

Numerische Methoden – Übungsaufgaben

Vollständig aus dem Vorlesungsskript “*Numerical Methods*” (Prof. Dr.-Ing. Mark Schutera) extrahiert · ins Deutsche übersetzt

Diese Sammlung enthält *alle* im Skript vorkommenden Aufgaben und durchgerechneten Beispiele – Handrechnungen, Code-/Maschinen-Aufgaben, Margin-Aufgaben, Denkanstöße und Selbstreflexionsfragen – über alle sieben Kapitel. Lösungswege: siehe **loesungen**.

Kapitel 1 – Einstieg in die Numerischen Methoden

1.1 – Minimum der Sinusfunktion [Handrechnung]

Bestimmen Sie das Minimum der Funktion $\sin(x)$ auf dem Intervall $[-3, 1]$, indem Sie die Funktion mit Schrittweite h diskretisieren, die Funktionswerte an den Gitterpunkten auswerten und den kleinsten auswählen. Vergleichen Sie mit dem analytisch exakten Minimum.

1.2 – Grid-Search [Handrechnung]

Führen Sie eine Grid-Search für w und b mit Schrittweite 2,5 über den Bereich $w, b \in \{0; 2,5; 5; 7,5; 10\}$ durch. Das zugrunde liegende System ist $\begin{pmatrix} 2 & 1 \\ 5 & 1 \end{pmatrix} \begin{pmatrix} w \\ b \end{pmatrix} = \begin{pmatrix} 11 \\ 13 \end{pmatrix}$. Berechnen Sie für jede Kombination den Fehler $\text{error}_1 = |2w + b - 11|$, $\text{error}_2 = |5w + b - 13|$. Werten Sie alle Kombinationen aus und wählen Sie das Paar (w, b) mit dem kleinsten akkumulierten Fehler.

1.3 – Enter the machine [Code]

Viele numerische Verfahren sind für die Umsetzung am Computer gedacht. Sie erhalten ein vorgehostetes Jupyter-Notebook mit einer Python-Vorlage für diese Übung. Nutzen Sie die Selbstreflexionsfragen, um sich beim Erkunden und Experimentieren mit der Implementierung leiten zu lassen.

Denkanstoß. Fällt Ihnen eine einfache Möglichkeit ein, das Verhältnis von Genauigkeit zu Rechenaufwand unseres Grid-Search-Beispiels zu verbessern?

Selbstreflexionsfragen

- Warum ist die Wahl der Schrittweite h beim Diskretisieren einer stetigen Funktion wichtig, und wie beeinflusst sie Genauigkeit und Rechenzeit?
- Wie beeinflusst die Wahl des Intervalls $[a, b]$ die Ergebnisse der Diskretisierung und die Lage der numerisch gefundenen Extrema?
- Was sind die wesentlichen Unterschiede zwischen einer stetigen Funktion und ihrer diskretisierten Version, und welche Implikationen ergeben sich daraus?

Kapitel 2 – Gleitkomma-Arithmetik

2.1 – Rundungsfehler [Handrechnung]

Bekommen wir an einem einfachen Beispiel ein Gefühl für den Rundungsfehler. Betrachten Sie die Division von 10 durch 3. Schreiben Sie die Dezimalentwicklung aus und bestimmen Sie den Rundungsfehler beim Abschneiden nach drei Nachkommastellen.

2.2 – 2-Bit-Mantisse & Maschinenepsilon [Handrechnung]

Eine unnormalisierte Mantisse hat 2 Bit, der Signifikand ist $0.xx_2$. Bestimmen Sie alle darstellbaren Werte und das zugehörige Maschinenepsilon $\varepsilon_{\text{mach}} = 2^{-t}$. Was bedeutet das für relative bzw. absolute Präzision?

2.3 – Auslöschung von Hand [Handrechnung]

Zwei Zahlen werden mit 8 signifikanten Stellen gespeichert. **(a)** $a = 12345678,5$, $b = 12345678,0$: bestimmen Sie $\tilde{a}, \tilde{b}$ und $\tilde{a} - \tilde{b}$ und vergleichen Sie mit $a - b$. **(b)** $a = 12345678,4$, $b = 12345678,0$: ebenso. Was beobachten Sie beim relativen Fehler?

2.4 – Festkomma-Skalierung [Handrechnung]

Sie speichern Zahlen aus $[-1000, 1000]$ in einem 32-Bit-Integer mit Skalierungsfaktor 10^6 . Wie wird 1,234567 gespeichert? Welche Präzision (kleinste darstellbare Differenz) und welcher maximale Rundungsfehler ergeben sich?

2.5 – Maschinenepsilon (hands on) [Code]

Überlegen Sie, wie Sie das Maschinenepsilon eines beliebigen Systems für ein bestimmtes Gleitkommaformat per

Programm bestimmen würden. Rufen Sie sich die Definition in Erinnerung und schreiben Sie Ihre Überlegungen als Pseudocode auf. Was würde ein solches Programm auf einem Festkomma- gegenüber einem Gleitkommasystem zeigen? Probieren Sie verschiedene Datentypen am Rechner aus. Wenn Sie einen Taschenrechner bauen würden – welchen Datentyp würden Sie wählen und warum?

2.6 – Auslöschung demonstrieren (hands on) [Code]

Überlegen Sie, wie Sie Auslöschung (loss of significance) an Ihrem Rechner demonstrieren würden. Schreiben Sie Ihre Überlegungen als Pseudocode auf, bevor Sie weiterlesen.

2.7 – Argumentation [Konzept]

Überlegen Sie, wie das Berechnen von $f(\epsilon) = \frac{1}{a + \epsilon - b}$ für kleines ϵ und $a = b$ dies leisten würde. Was erwarten Sie zu sehen, wenn ϵ sehr klein ist?

Randnotiz: Was passiert für $a > b$? Denken Sie in signifikanten Stellen.

Denkanstoß. Fallen Ihnen Metriken für numerische Verfahren ein, basierend auf den Approximationen und Fehlern, die wir besprochen haben?

Selbstreflexionsfragen

- Wie ist die Gleitkommadarstellung aufgebaut, und was sind ihre Komponenten?
- Was ist das Maschinenepsilon, und wie hängt es mit der numerischen Genauigkeit zusammen?
- Wie wirken sich diese Konzepte in der Praxis auf numerische Berechnungen aus?

Kapitel 3 – Fehleranalyse

3.1 – Minimum der Sinusfunktion (Eigenschaften) [Handrechnung]

Bestimmen Sie das Minimum von $\sin(x)$ auf $[-3, 1]$ noch einmal. Diskretisieren Sie mit zwei Schrittweiten $h = 1$ und $h = 0,2$. Nehmen Sie an, dass die Eingabedaten x durch Messfehler um $\tilde{x} = x \pm 0,25$ gestört sind (zwei Dezimalstellen genau). Analysieren Sie **Kondition**, **Stabilität**, **Konsistenz** und **Konvergenz** des numerischen Verfahrens in beiden Fällen und quantifizieren Sie sie mit den im vorigen Abschnitt angegebenen Formeln.

3.2 – Stabilität und Kondition [Beweis]

Zeigen Sie: Die Stabilität entspricht der Kondition des Systems für $h \rightarrow 0$.

3.3 – Compute-getriebene Fehleranalyse (hands on) [Code]

Brute-forcen wir uns durch die Fehleranalyse beim Lösen des Zwei-Gleichungs-Systems aus Kapitel 1. Bestimmen und optimieren Sie Kondition, Stabilität, Konsistenz und Konvergenz der numerischen Lösung, indem Sie das numerische Verfahren optimieren.

Selbstreflexionsfragen

- Wie gut konditioniert ist das Zwei-Gleichungs-System verglichen mit der numerischen Lösung des Minimums der Sinusfunktion?
- Konvergiert die Stabilität beim Zwei-Gleichungs-System mit kleineren Schrittweiten? Wogegen?
- Wie wirken sich Störungen der Eingabedaten auf die Lösung aus? Gibt es ein Zusammenspiel mit der Schrittweite?
- Vergleichen Sie Konvergenz mit Stabilität und Konsistenz. Können Sie Ihre Beobachtungen in Begriffen von Bias und Varianz ausdrücken?
- Beobachten Sie für immer kleinere Schrittweiten eine Grenze der Genauigkeit? Wenn ja, warum?
- Welches weitere Problem beobachten Sie beim Verfeinern der Schrittweite?

Denkanstoß. Bisher haben wir Brute-Force-Lösungen verwendet. Fällt Ihnen eine Möglichkeit ein, Brute-Force-Methoden zu verfeinern, um mit weniger Aufwand bessere Ergebnisse zu erzielen?

Kapitel 4 – Newton-Verfahren

4.1 – Newton für $\sqrt{2}$ [Handrechnung]

Berechnen Sie $\sqrt{2}$ mit dem Newton-Verfahren, ausgehend von $x_0 = 2$ (also die Nullstelle von $f(x) = x^2 - 2$). Führen Sie einige Iterationen durch und beobachten Sie, wie sich der Fehler pro Schritt etwa quadriert.

4.2 – Newton vs. Grid-Search für $\sin(x) = 0$ [Handrechnung]

Vergleichen Sie Grid-Search und Newton-Verfahren beim Finden der Nullstelle von $\sin(x) = 0$ auf $[2,5; 4]$ (Lösung nahe π). (a) Grid-Search mit Schrittweite $h = 0,3$. (b) Newton ($f' = \cos$), Start $x_0 = 3,5$. (c) Vergleichen Sie Genauigkeit und Anzahl der Auswertungen.

4.3 – Denksportaufgabe: Versagensmodi [Schaubild]

Das Newton-Verfahren kann auf verschiedene Weisen versagen. Visualisieren Sie die Versagensmodi im Schaubild von $f(x) = x^3 - x$.

4.4 – Newton von Hand [Handrechnung]

Finden Sie die Nullstelle von $f(x) = x^3 - x = 0$, indem Sie das Newton-Verfahren geometrisch und anschließend von Hand anwenden. Die Nullstellen liegen bei $x = -1, 0, 1$. Suchen wir die Nullstelle bei $x = 1$, ausgehend von $x_0 = 1,4$.

4.5 – Sekantenverfahren von Hand [Handrechnung]

Wenden Sie das Sekantenverfahren auf dasselbe Problem mit $x_0 = 1,2$, $x_1 = 1,4$ an. Wieder zuerst geometrisch, dann von Hand.

4.6 – Geometrische Fehlersuche [Handrechnung]

Finden Sie Startpunkte x_0 für verschiedene Versagensmodi: einen, bei dem Newton wegen verschwindender Ableitung versagt, einen, bei dem es zu einer Nicht-Ziel-Nullstelle konvergiert, und einen, bei dem es zwischen Punkten oszilliert.

4.7 – Enter the machine [Code]

Betrachten wir Konvergenz und Konvergenzraten von Grid-Search, Newton- und Sekantenverfahren in Python. Experimentieren Sie mit verschiedenen Startpunkten und beobachten Sie die Versagensmodi – zuerst geometrisch, dann analytisch, dann im Code.

Selbstreflexionsfragen

- Was ist der Hauptvorteil des Newton-Verfahrens gegenüber der Grid-Search?
- Warum genügt eine lineare Approximation zum Finden von Nullstellen? Was sagt die Taylor-Entwicklung darüber?
- Warum ist quadratische Konvergenz so mächtig? Wie viele Iterationen bräuchte Newton, um von Fehler 10^{-1} auf 10^{-16} zu kommen?
- In welchen Situationen würden Sie das Sekantenverfahren vorziehen? Warum nicht in anderen?

Denkanstoß. Wie können wir sicherstellen, dass wir die globale Lösung finden und nicht nur eine lokale?

Kapitel 5 – Globale Optimierung

5.1 – Newton auf Shekel, weiterer Startpunkt [Code]

Wenden Sie das Newton-Verfahren auf die 1D-Shekel-Funktion von verschiedenen Startpunkten an und beobachten Sie, wohin es konvergiert. Probieren Sie selbst einen weiteren Startpunkt und diskutieren Sie das allgemeine Verhalten.

Randnotiz: Wie würden Sie die Update-Regel abändern, um stattdessen Maxima zu finden?

5.2 – Random Restarts [Handrechnung]

Angenommen, Sie optimieren eine Funktion mit 5 lokalen Minima, und das Einzugsgebiet des globalen Minimums deckt 15% des Suchraums ab ($p = 0,15$). Wie groß ist die Wahrscheinlichkeit, das globale Minimum mit $K = 10$ Random Restarts zu finden? Nutzen Sie $P = 1 - (1 - p)^K$ und lösen Sie nach K auf: $K = \frac{\ln(1-P)}{\ln(1-p)}$. Wie viele Restarts brauchen wir für 99%? Berechnen Sie dann K für $p = 0,05$ und $p = 0,01$ beim selben 99%-Ziel.

5.3 – Eigene 1D-Shekel-Funktion [Code]

Entwerfen Sie an Ihrem Rechner Ihre eigene 1D-Shekel-Funktion (anfangs einfach, später komplexer). Wenden Sie die lokalen Optimierungsverfahren an und zeigen Sie, wo Newton (erneut) versagt. Untersuchen Sie verschiedene x_0 und wie sich das Konvergenzverhalten ändert.

5.4 – Die Suche zu Minima lenken [Konzept + Code]

Erinnern Sie sich an den $|f''|$ -Trick, der das Vorzeichen der Krümmung im Nenner umkehrt und den Schritt zwingt, stets bergab zu gehen. Überlegen Sie, was das geometrisch für die Tangentenkonstruktion bedeutet, und probieren Sie es im Code.

5.5 – Einzugsgebiet (Basin of Attraction) [Schaubild]

Betrachten Sie die Plots der Shekel-Funktion, markieren Sie die Einzugsgebiete jedes lokalen Minimums und schätzen Sie ihre relativen Größen. Welches ist das globale Minimum? Wie hängt das mit der Wahrscheinlichkeit zusammen, es per Random Restarts zu finden?

5.6 – Lokalen Minima entkommen [Code]

Setzen Sie Random Restarts und Basin Hopping auf Ihrer 1D-Shekel-Funktion ein und vergleichen Sie, wie viele Funktionsiterationen jede Methode bis zum globalen Minimum braucht. Wie anfällig sind sie fürs Hängenbleiben? Wie beeinflusst die Schrittweitenverteilung die Performance des Basin Hopping?

5.7 – Nicht-konvexe Becken [Konzept]

Shekel ist im Allgemeinen nicht-konvex, die einzelnen Becken jedoch schon; notieren Sie eine Funktion, bei der die Einzugsgebiete nicht so leicht abzugrenzen sind.

5.8 – Adaptive Schrittweite [Konzept + Code]

Fällt Ihnen ein geschickter Weg ein, die Schrittweitenverteilung während der Optimierung automatisch anzupassen? Was wäre eine gute Heuristik? Wie könnten Sie katastrophale Sprünge verhindern? Implementieren Sie es.

5.9 – Noise and landscape engineering [Konzept]

Die Loss-Landschaft ist nicht statisch, sondern soll geformt werden. Überlegen Sie, wie das Approximieren der Loss-Landschaft auf Mini-Batches (unterschiedliche Auswahl von Punkten) wirkt.

5.10 – Code-Übung: Restarts vs. Basin Hopping [Code]

Implementieren Sie Random Restarts und Basin Hopping auf der 1D-Shekel-Funktion. Vergleichen Sie, wie viele Funktionsauswertungen jede Methode braucht, um das globale Minimum bei $x \approx 4$ zu finden.

5.11 – Einzugsgebiete kartieren [Handrechnung]

Betrachten Sie die 1D-Shekel-Funktion; ein lokaler Optimierer konvergiere stets zum nächstgelegenen Foxhole. (a) Skizzieren Sie (grob) die Einzugsgebiete für die drei Foxholes bei $x = 0, 4, 7$. Wo liegen die Basin-Grenzen? (b) Wenn Basin Hopping einen Sprung $\delta \sim \text{Uniform}(-3, 3)$ ausgehend vom lokalen Minimum $x^\circ = 7$ verwendet – wie groß ist die Wahrscheinlichkeit, dass ein einzelner Sprung im Becken des globalen Minimums landet? (c) Warum könnte Basin Hopping das globale Minimum hier schneller finden als Random Restarts?

Selbstreflexionsfragen

- Warum versagt lokale Optimierung auf nicht-konvexen Landschaften wie Shekels Foxholes?
- Wie skaliert der erforderliche Aufwand mit der Größe des globalen Beckens bei Random Restarts?
- Wie unterscheidet sich Basin Hopping von Random Restarts?
- Was sagt uns $f''(x_n)$ über die Loss-Landschaft, und wie können wir es nutzbar machen?
- Wie hilft das Approximieren der Loss-Landschaft mit verrauschten Schätzungen (Mini-Batches), lokalen Optima zu entkommen?
- Wählen Sie x_0 im Becken eines lokalen Minimums; finden Sie Wege, mit denen Ihr Verfahren entkommen kann – wie wirksam ist welcher?

Denkanstoß. Was, wenn wir einer hochfrequenten Version von Shekels Foxholes gegenüberstehen? Welches Problem tritt dabei auf?

Kapitel 6 – Numerische Integration

6.1 – Exakter Wert [Handrechnung]

Berechnen Sie das Integral von $\sin(x)$ von -2 bis -1 exakt – als Referenzwert zur Fehlerquantifizierung der folgenden Methoden.

6.2 – Mittelpunktsregel von Hand [Handrechnung]

Berechnen Sie $\int_{-2}^{-1} \sin(x) dx$ mit der Mittelpunktsregel mit $n = 1$ Intervall.

6.3 – Mittelpunktsregel mit mehr Intervallen [Handrechnung]

Berechnen Sie die Mittelpunktsregel mit $n = 2$ Intervallen (Mittelpunkte bei $-1,75$ und $-1,25$).

Randnotiz: Berechnen Sie die Approximation auch an den Endpunkten des Intervalls und vergleichen Sie den Fehler.

6.4 – Trapezregel von Hand [Handrechnung]

Berechnen Sie mit der Trapezregel $\int_{-2}^{-1} \sin(x) dx$ mit $n = 2$ Intervallen. Die Stützstellen sind $-2, -1,5, -1$.

6.5 – Simpson-Regel von Hand [Handrechnung]

Berechnen Sie mit der Simpson-Regel $\int_{-2}^{-1} \sin(x) dx$ mit $n = 2$ Intervallen. Die Stützstellen sind $-2, -1,5, -1$.

Randnotiz: Probieren Sie mehr Intervalle und prüfen Sie sich mit zusammengesetzten Simpson- und Trapezregeln.

6.6 – Monte Carlo von Hand [Handrechnung]

Schätzen Sie $\int_{-2}^{-1} \sin(x) dx$ per Monte Carlo mit $N = 4$ Zufallsstichproben (etwa der Rechenaufwand der Trapezmethode). Die Stichproben sind $X_1 = -1,95, X_2 = -1,55, X_3 = -1,15, X_4 = -1,05$.

6.7 – Monte Carlo: Fehler & N erhöhen [Konzept]

Bestimmen Sie Stichprobenvarianz und Standardfehler des Monte-Carlo-Schätzers. Was passiert, wenn Sie N auf 16 erhöhen? Erläutern Sie kurz, warum der Fehler des Monte-Carlo-Schätzers dimensionsunabhängig ist.

6.8 – Simpson-Gewichte via Lagrange [Herleitung]

Leiten Sie die Quadraturgewichte der Simpson-Regel her, indem Sie die Lagrange-Basispolynome über $[a, b]$ integrieren.

Randnotiz: Beginnen Sie mit $x = 0$ und sehen Sie, wie die Linearfaktoren ausfallen. Was passiert, wenn Sie eine lineare Funktion mit einer quadratischen approximieren?

6.9 – Höhere Dimensionen (on your machine) [Code]

Implementieren Sie Monte-Carlo-Integration für ein d -dimensionales Integral (z. B. eine multivariate Gauß-Funktion über einen Hyperwürfel). Erhöhen Sie d langsam und beobachten Sie das Fehlerverhalten und wie gitterbasierte Quadratur rechnerisch undurchführbar wird.

Selbstreflexionsfragen

- Wie vergleichen sich die Fehler von Mittelpunkts-, Trapez-, Simpson-Regel und Monte Carlo?
- Warum hat die Mittelpunktsregel eine bessere Genauigkeit als die Links- oder Rechts-Endpunkt-Regel?
- Wie verbessern zusammengesetzte Methoden die Genauigkeit, und was ist der Trade-off?
- Warum versagen deterministische Quadraturmethoden in hohen Dimensionen?
- Inwiefern ist die Mittelpunktsregel ein Spezialfall der Monte-Carlo-Schätzung?
- Was ist die Intuition dahinter, dass Monte Carlo in hohen Dimensionen nicht “explodiert”?
- Wann würden Sie die Simpson-Regel gegenüber Monte Carlo verwenden?
- Wie hängt der Mini-Batch-Mittelwert in SGD mit der Monte-Carlo-Schätzung zusammen?
- Wie hilft die Gradientenschätzung in SGD, lokalen Minima zu entkommen?

Denkanstoß. Warum werden hochgradige Quadraturmethoden instabil, wenn sie Polynome niedrigen Grades approximieren?

Kapitel 7 – Modelltraining (Training a Model from First Principles)

7.1 – Loss bei Initialisierung [Handrechnung]

Lineare Regression $\hat{y}(x) = wx + b$, MSE-Loss $L(w, b) = \frac{1}{n} \sum_i (wx_i + b - y_i)^2$, Initialisierung $w = 0, b = \bar{y}$. Zeigen Sie, dass $L(0, \bar{y}) = \text{Var}(y)$. Mit $\bar{y} = 231/14 = 16,5$ und $\sum_i (y_i - \bar{y})^2 = 1401,5$: berechnen Sie den erwarteten ersten Loss-Wert. Was bedeutet ein gedruckter Wert von 5000 bzw. 0,0001?

7.2 – Overfitting als Sanity-Check [Konzept]

Ein Grad-4-Polynom wird durch zwei Trainingspunkte (Italien (4,0; 4), UK (9,5; 21)) gelegt. Wie viele Null-Loss-Kurven gehen durch die zwei Punkte, und was sagt diese Anzahl über die Geometrie der Verlustlandschaft aus?

7.3 – Mini-Batch-SGD [Konzept]

In welchem präzisen Sinn ist Mini-Batch-SGD ein Monte-Carlo-Schätzer (was wird worüber gemittelt)? Ab welchem n kippt der Engpass von voll-batch zu mini-batch, gegeben der Fehler skaliert mit $1/\sqrt{B}$?

7.4 – Mehrere Random Seeds [Konzept]

Für das Modell $\hat{y}(x) = w^2x + b$: Warum ist der Ursprung ein Sattel statt eines Minimums, und warum ist die Null-Initialisierung trotzdem eine Falle? Wozu dient das Training aus mehreren Random Seeds?

7.5 – Quantisierung für Inferenz [Konzept]

Auf dem Schokoladen-Datensatz erreichte die Integer-Präzision fast denselben Fehler wie volle Präzision. Warum versenden Produktionsmodelle dennoch in float32? Warum kann ein in hoher Präzision trainiertes Modell in niedriger Präzision eingesetzt werden, während Training in niedriger Präzision scheitert?

Selbstreflexionsfragen

- Warum entdeckt das Treiben des Loss auf 0 bei einem überparametrisierten Modell Bugs in der Pipeline statt im Optimierer?
- Was sagt ein gedruckter erster Loss von 5000 bzw. 0,0001 (statt ≈ 100) aus?
- In welchem präzisen Sinn ist Mini-Batch-SGD ein Monte-Carlo-Schätzer?
- Warum ist für $\hat{y} = w^2x + b$ der Ursprung ein Sattel, und warum ist Null-Init eine praktische Falle?
- Warum lässt sich ein hochpräzise trainiertes Modell niedrigpräzise einsetzen, während Training in niedriger Präzision scheitert?